Practical No. 3

Title: Implementing Hashing on Telephone Book Database.

Problem Statement:

Consider the telephone book database of N clients. Make use of a hash table implementation to quickly look up a client's telephone number. Make use of linear probing, double hashing and quadratic collision handling techniques.

Aim:

Write C/C++ program to make use of a hash table implementation to quickly look up a client's telephone number from telephone book database of N clients. Make use of -

- Linear Probing
- Quadratic Probing
- Double Hashing

Objectives:

- To study concept of Hashing.
- To understand implementation of various collision handling techniques like Linear Probing, Quadratic Probing and Double Hashing.
- To find record quickly using Hash Function.

Prerequisites:

- Knowledge of C and C++ Programming, Object Oriented Programming.
- Knowledge of Arrays and Searching.

Outcomes:

- Implement use of Array.
- Implement various collision handling techniques.
- Implement concept of hash table.

Software & Hardware requirements:

- Open Source C/C++ Programming tools like G++/GCC or Eclipse.
- 64-bit Open source Linux or its derivative.

Theory Concept:

Hashing

Hashing is finding an address where the data is to be stored as well as located using a key with the help of the algorithmic function. Hashing is the process of mapping data to a specific index in a data structure (Hash Table). Hashing is the process of converting a given key into another value. **Hashing** is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access. Hashing is a method of directly computing the address of the record with the help of a key by using a suitable mathematical function.

Key

A Key can be anything string or integer which determines an index or location for storage of an item in a data structure.

Hash Function

A function that maps a key into the range [0 to Max − 1] which gives result which is used as an index (or address) to hash table for storing and retrieving record. It receives the input key and returns the index of an element in an array. It is an algebraic function denoted by f(x) or f(k) or F(x) or F(k) or h(x) or h(k) or H(x) or H(k) where x is value which is used as key and k is key.

Format: Hash_Function = Key/Value Operator Table_Size/Value.

Example: H(x) = x % 10 is the Hash Function in below:

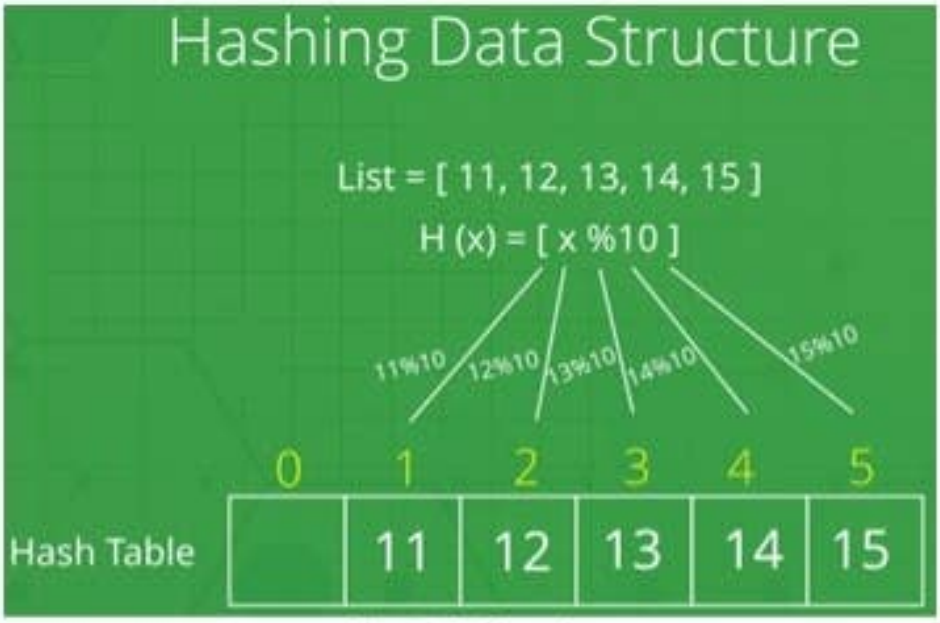


Fig. Hash Function

Bucket

Bucket is an index position in hash table that can store more than one record.

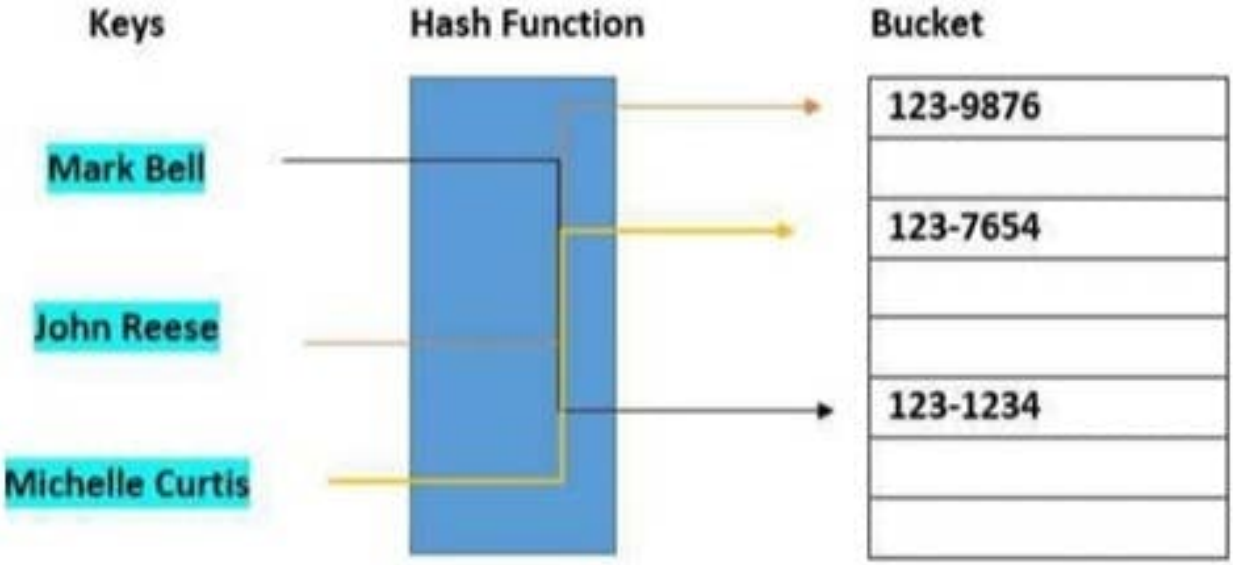


Fig. Bucket

Hash Table

A hash table is a data structure that stores records in an array. It used to stores data in key, value pairs.

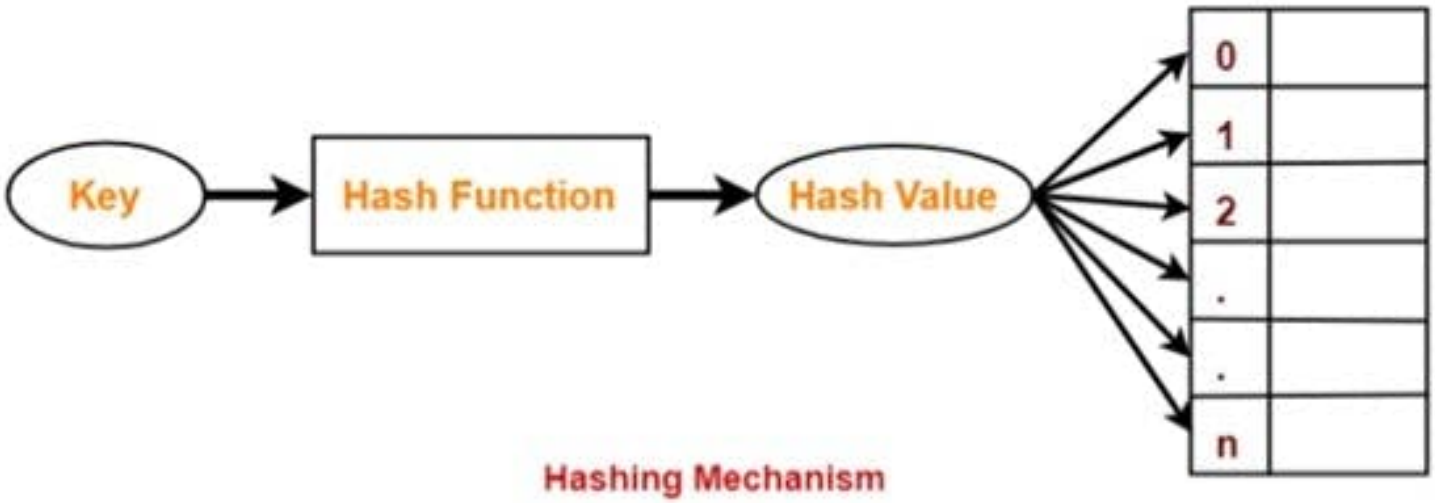
Index	Value
0	value_1
1	value_2
2	value_3
3	value_4

Key	Value
00	30
01	100
02	50

Fig. Hash Table

Hashing Mechanism

In Hashing, a key is used by Hash Function which converts key into index called Hash Value and stores key at respective index into Bucket.



Hashing Mechanism

Collision

The result of two or more keys hashing into the same address or same index is called collision.

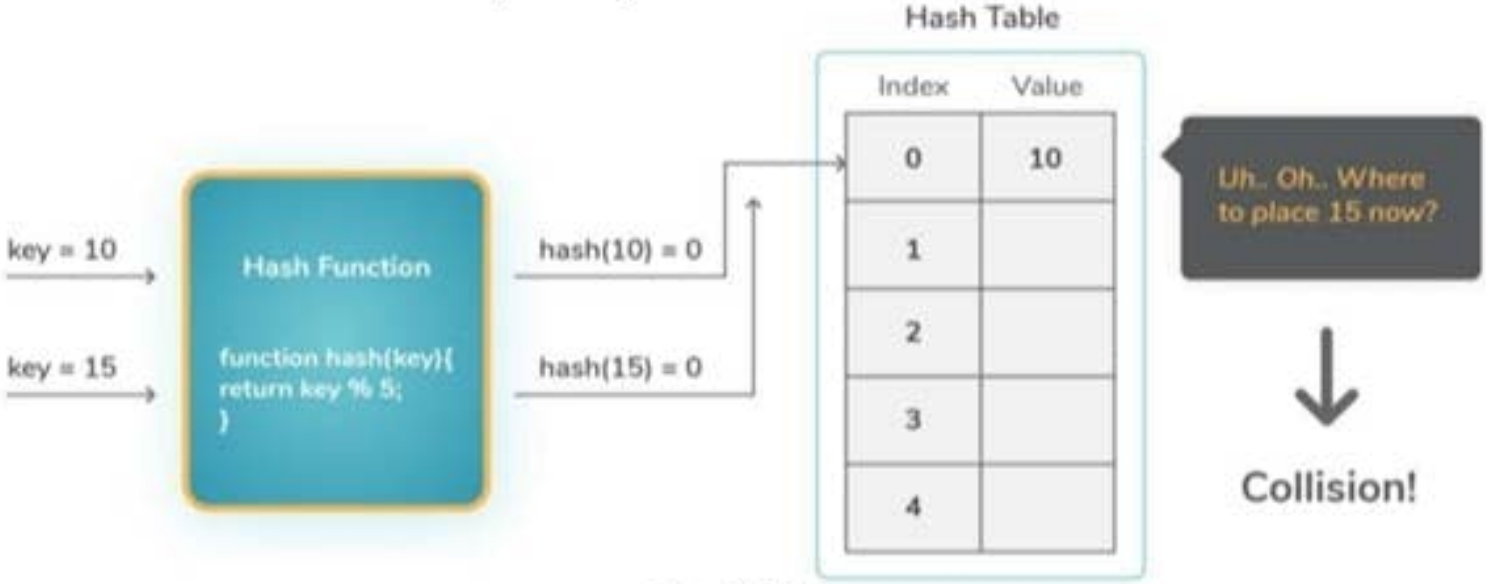


Fig. Collision

Types of Hash Function

There are many ways of calculating the hash function. Some common ways are:
1. Division-Modulo method
2. Digit Folding method
3. Mid Square method

Division-Modulo Method

In Division-Modulo Method, the hash function divides the value k by number or size M and then uses the remainder obtained.
Formula: h(k) = k mod M.
Example: k = 12345, M = 95. h(12345) = 12345 mod 95 = 90, h(26) = 26 % 10 = 6

Digit Folding Method

This method involves two steps: Divide the key-value k into a number of parts i.e. k1, k2, k3, …. ,kn , where each part has the same number of digits except for the last part that can have lesser digits than the other parts. Add the individual parts. The hash value is obtained by ignoring the last carry if any.
Formula: k = k1, k2, k3, k4, ….. , kn. s = k1+ k2 + k3 + k4 +….+ kn. h(K)= s
Example: k = 12345. k1 = 12, k2 = 34, k3 = 5. s = k1 + k2 + k3 = 12 + 34 + 5 = 51. h(K) = 51

Mid Square Method

It involves two steps to compute the hash value: Square the value of the key k i.e. k^2 and Extract the middle digits as the hash value.
Formula: h(k) = Middle Digits of k x k.
Example: k = 60. k x k = 60 x 60 = 3600. h(60) = 60.

Overflow

Overflow is the condition where hash value is inserted into already occupied bucket. It means filling the bucket more than it's capacity. Example: If bucket capacity is 3 and 4th Value is inserted into the bucket, Overflow condition occurs.

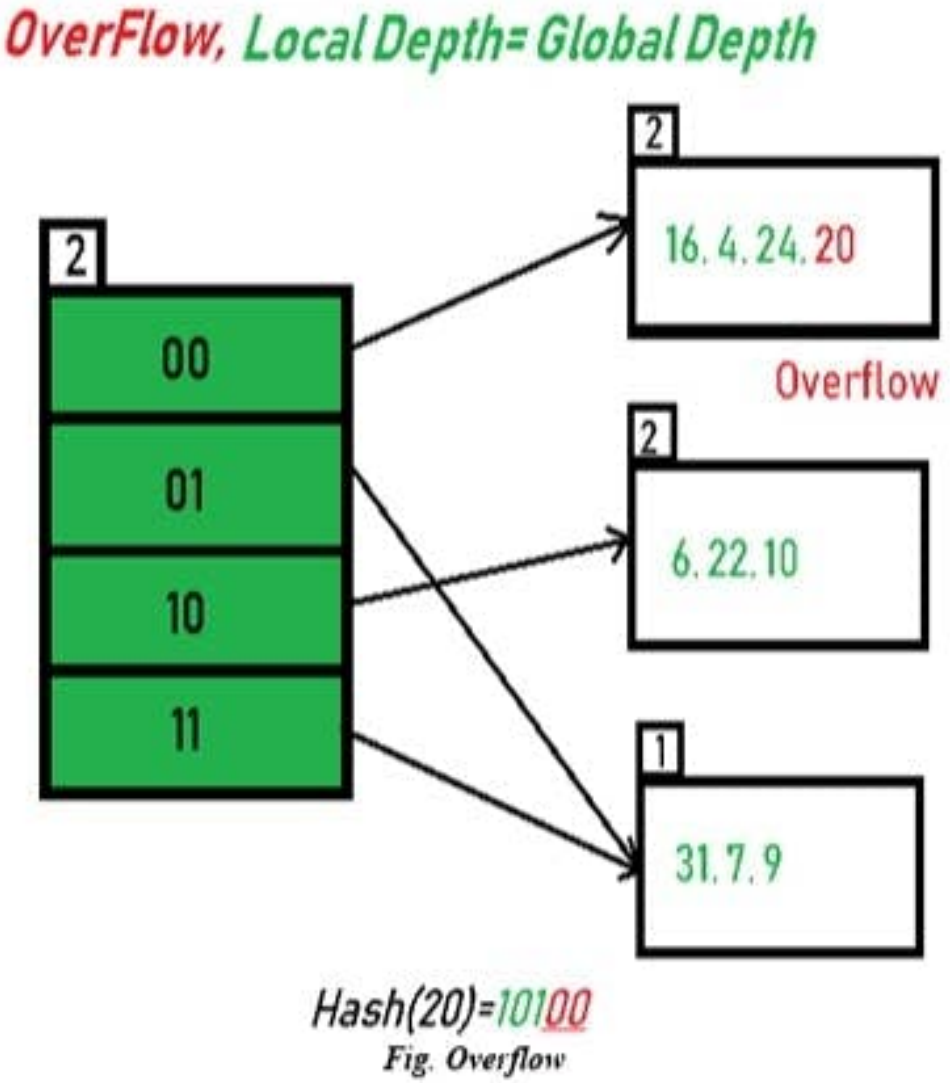


Fig. Overflow

Collision Resolution Techniques/Collision Handling Techniques

When the two or more different values have the same value, then the problem occurs between the two values is known as a collision. As in example, when bucket is full, there is no space which results in collision between 2 or more keys or values. To avoid collision, Collision Resolution/Handling Techniques are used. Following are the Collision Resolution/Handling Techniques:

1. Open Hashing or Closed Addressing
2. Closed Hashing or Open Addressing

Open Addressing/Closed Hashing

Open addressing is a collision resolution technique used in hash tables where all key-value pairs are stored directly within the hash table itself. When a collision occurs (two keys hash to the same index), open addressing uses a probing strategy to find an alternative empty slot for the new key-value pair.

Probing

Probing in hashing refers to the method used to find an appropriate slot in a hash table when a collision occurs.

Types of Open Addressing/Closed Hashing Collision Resolution/Handling Techniques

In Open Addressing/Closed hashing, three techniques are used to resolve the collision:
1. Linear Probing
2. Quadratic Probing
3. Double Hashing

Linear Probing

When the collision occurs by mapping a new key to the cell already occupied by another key, then linear probing technique searches for the closest free locations and adds a new key to that empty cell. Searching is performed sequentially, starting from the position where the collision occurs till the empty cell is not found. If linear probing reaches end of the table, searching starts from the beginning of the table.

Formula: $h(x) = x \% m = u$. When Collision occurs, $h(x)$ becomes $h(x) = (u + i) \% m$, where m = table size/value, u = value obtained by original hash function, x = key and i = increment (usually $i = 1$ but when end of table is reached, search starts from beginning where $i \neq 1$).

Example: 2,3,5,22,34. $h(x) = x \% 5$.

Table Size m is 5. Keys are 2,3,5,22,34.
Step 1: $h(2) = 2 \% 5 = 2$. Key is placed at 2nd index. Insert 2.
Step 2: $h(3) = 3 \% 5 = 3$. Key is placed at 3rd index. Insert 3.
Step 3: $h(5) = 5 \% 5 = 5$. Key is placed at 0th index. Insert 5.

Index	Key or Bucket
0	5
1	
2	2
3	3
4	

← Inserting 22, Overflow, Collision, Bucket is Full.

Step 4: $h(22) = 22 \% 5 = 2$. Key is placed at 2nd index. Collision, Bucket is Full. Find next empty space sequentially. Empty Space found at 4th index. Insert 22.

Index	Key or Bucket
0	5
1	
2	2
3	3
4	22

← Inserting 34, Collision, Bucket is Full.

Step 5: $h(34) = 34 \% 5 = 34$. Key is placed at 4th index. Collision, Bucket is Full. Find next empty space sequentially. Reached End of the Table/List. Search from Beginning. Empty Space found at 1st index. Insert 34.

Index	Key or Bucket
0	5
1	34
2	2
3	3
4	22

Quadratic Probing

When the collision occurs by mapping a new key to the cell already occupied by another key, then quadratic probing technique adds a new key to the empty cell using quadratic function/polynomial.

Formula: $h(x) = x \% m = u$. When Collision occurs, $h(x)$ becomes $h(x) = (u + i^2) \% m$, where m = table size/value, u = value obtained by original hash function, x = key and i = increment.

Example: 2,3,5,22,34. $h(x) = x \% 5$.

Table Size m is 5. Keys are 2,3,5,22,34.

Step 1: $h(2) = 2 \% 5 = 2$. Key is placed at 2nd index. Insert 2.
Step 2: $h(3) = 3 \% 5 = 3$. Key is placed at 3rd index. Insert 3.
Step 3: $h(5) = 5 \% 5 = 5$. Key is placed at 0th index. Insert 5.

Index	Key or Bucket
0	5
1	
2	2
3	3
4	

← Inserting 22, Overflow, Collision, Bucket is Full.

Step 4: $h(22) = 22 \% 5 = 2$. Key is placed at 2nd index. Collision, Bucket is Full. $u = 2$. $h(x) = (u + i^2) \% 5$.
$i = 1$, $h(22) = (2 + 1^2) \% 5 = (2 + 1) \% 5 = 3 \% 5 = 3$. Key is placed at 3rd index. Collision, Bucket is Full.
$i = 2$, $h(22) = (2 + 2^2) \% 5 = (2 + 4) \% 5 = 6 \% 5 = 1$. Key is placed at 1st index. Insert 22.

Index	Key or Bucket
0	5
1	22
2	2
3	3
4	

Step 5: $h(34) = 34 \% 5 = 34$. Key is placed at 4th index. Insert 34.

Index	Key or Bucket
0	5
1	22
2	2
3	3
4	34

Double Hashing

Double hashing uses the idea of applying a second hash function to key when a collision occurs.

Example: $h(x) = x \% m$. When Collision occurs, $h(k)$ becomes $h_1(k)$.
$h_2(x) = c - (x \% m)$ or $h_2(x) = 1 + [x \% (m-1)]$
$h(x) = [h_1(x) + i\, h_2(x)] \% m$, where m = table size/value, c = constant, x = key and i = increment.

Consider keys = 2,3,5,22,34. $h(x) = x \% 5$.
Table Size m is 5. Keys are 2,3,5,22,34.
Since c is not given, either consider any value as c or consider $h_2(x) = 1 + [x \% (m-1)]$.
$h_2(x) = 1 + [x \% (m-1)] = 1 + [x \% (5-1)] = 1 + (x \% 4)$.
$h(x) = [h_1(x) + i\, h_2(x)] \% m = h(x) = [h_1(x) + i\, h_2(x)] \% 5$
Step 1: $h(2) = 2 \% 5 = 2$. Key is placed at 2nd index. Insert 2.
Step 2: $h(3) = 3 \% 5 = 3$. Key is placed at 3rd index. Insert 3.
Step 3: $h(5) = 5 \% 5 = 5$. Key is placed at 0th index. Insert 5.

Index	Key or Bucket
0	5
1	
2	2
3	3
4	

Inserting 22, Overflow, Collision, Bucket is Full.

Step 4: $h(22) = 22 \% 5 = 2$. Key is placed at 2nd index. Collision, Bucket is Full. $h_1(22) = 2$.
$h_2(22) = 1 + (22 \% 4) = 1 + 2 = 3$. $i = 1$, $h(x) = [h_1(x) + i\, h_2(x)] \% 5 = [2 + 1 * 3] \% 5 = 5 \% 5 = 0$.
Key is placed at 0th index. Collision, Bucket is Full. Increment i.
$i = 2$, $h(x) = [h_1(x) + i\, h_2(x)] \% 5 = [2 + 2 * 3] \% 5 = [2 + 6] \% 5 = 8 \% 5 = 3$.
Key is placed at 3rd index. Collision, Bucket is Full. Increment i.
$i = 3$, $h(x) = [h_1(x) + i\, h_2(x)] \% 5 = [2 + 3 * 3] \% 5 = [2 + 9] \% 5 = 11 \% 5 = 1$.
Key is placed at 1st index. Insert 22.

Index	Key or Bucket
0	5
1	22
2	2
3	3
4	

Step 5: $h(34) = 34 \% 5 = 34$. Key is placed at 4th index. Insert 34.

Index	Key or Bucket
0	5
1	22
2	2
3	3
4	34

Searching in Hashing

To search any particular key, its hash value is obtained using the hash function used. Using the hash value, that bucket of the hash table is checked. If the required key is found, the key is searched. Otherwise, the subsequent buckets are checked until the required key or an empty bucket is found. The empty bucket indicates that the key is not present in the hash table.

Implementation in Program

Take input from user as number of clients. Consider them as N clients. Consider N as table size. Now use Hash Function to Begin Insertion in Hash Table. Use Linear Probing, Quadratic Probing and Double Hashing for Collision Handling in the Hash Table. Display Entries once all Phone numbers are added. Look Up a Telephone Number by Searching in the Hash Table. To implement a hash table in C, a dynamic array can be created using malloc() and sizeof(). In C++, the new operator can be used for dynamic allocation. Additionally, C++ STL containers such as vector (for dynamic arrays) and unordered_map (for built-in hash tables) can also be used.

Algorithm:

1: Create a hash table with maximum N size.

2: Define Hash function.

3: Read the telephone no & user's information to insert it into the hash table.

4: Insert telephone no value in the hash table.

5: If collision occurs, apply a separate chaining.

6: If a user requires inserting another data, go to step 4.

7: Display all the user's data.
8: Read the telephone no from the user to be found.
9: find the data from the hash table.
10: Display the data with minimum no. of comparisons.

Flowchart:

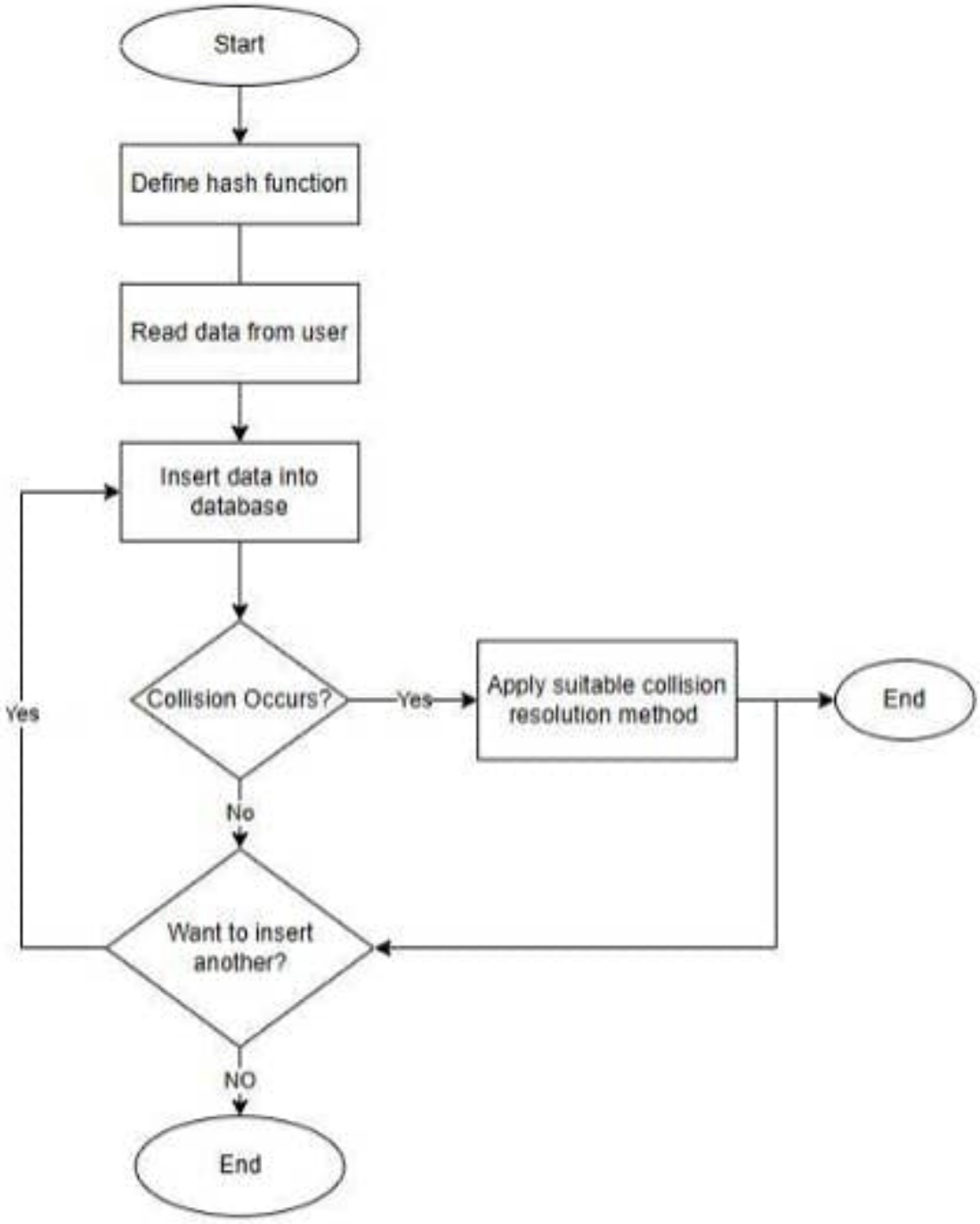


Fig. Flowchart for Hashing

Input: Telephone book database of N clients and their telephone numbers.
Output: Results based on hash table implementation of telephone book database of N clients by using collision handling techniques like Linear Probing, Quadratic Probing, Double Hashing, displaying them and finding client's telephone number.

Conclusion: Collision handling techniques like Linear Probing, Quadratic Probing and Double Hashing were studied, understood and implemented on program to make telephone book database of N clients using hash table, store their telephone numbers, display them and quickly look them up successfully.